

Skills Demonstration #3 - 30%:

Design, code and test an object oriented program that uses inheritance

Programming Description

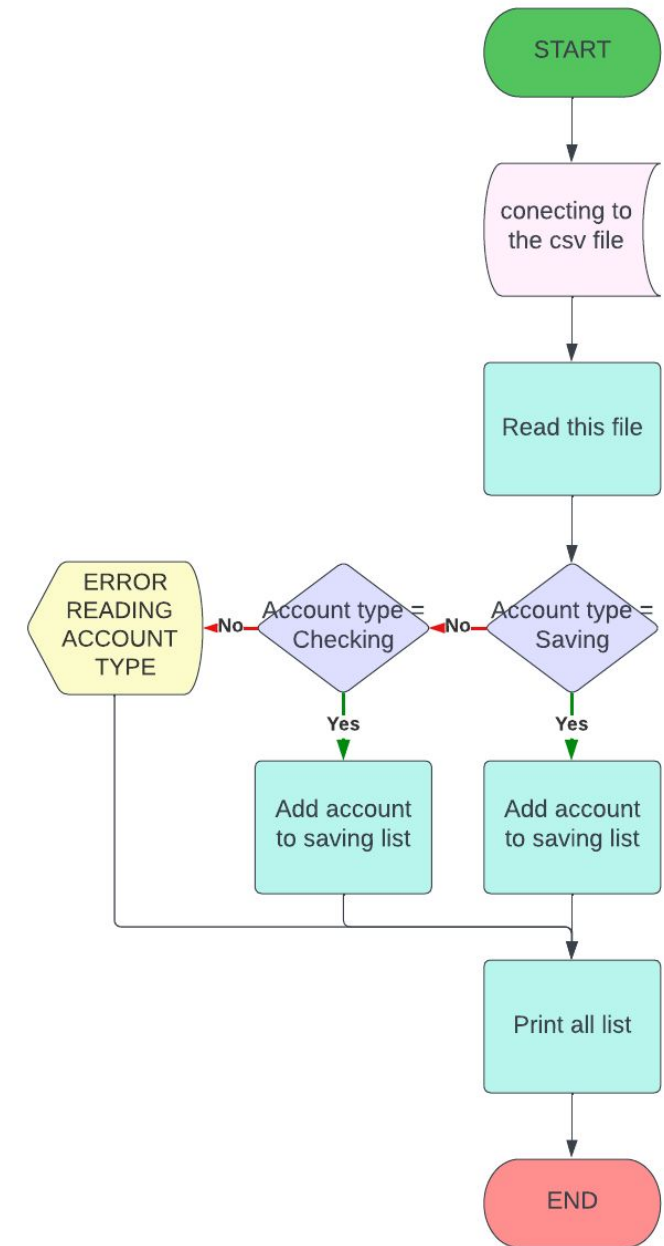
Banking app

- The BankAccountApp is a Java application for managing bank accounts. It contains three classes: Account, Checking, and Saving.
- The Account class is an abstract class that defines the base properties and methods for a bank account, including the account holder's name, PPS number, balance, account number, and interest rate. The Account class implements the IBaseRate interface, which contains a single method for setting the base interest rate for all bank accounts.
- The Checking and Saving classes are subclasses of the Account class. The Checking class adds properties for the debit card number and PIN, while the Saving class adds a property for the safety deposit box number and key.
- The BankAccountApp class contains the main method, which creates new Checking and Saving objects from a CSV file, and then displays information about each account. The CSV file contains information about new account holders, including their name, PPS number, account type, and initial deposit.

Flow chart

There are three classes in my program: Account, Checking, and Saving.

- Account is an abstract class that is the parent/super class for Checking and Saving classes. The Account class contains common properties and methods that both Checking and Saving accounts share, such as the account holder's name, PPS number, balance, account number, and methods for depositing, withdrawing, transferring, and printing the balance.
- The Checking and Saving classes are sub-classes that inherit the properties and methods of the Account class. In addition to the common properties and methods inherited from Account, each sub-class has its own unique properties and methods. For example, the Checking class has properties such as cardNumber and cardPIN, which are specific to a checking account, and the Saving class has a property safetyDepositBoxID that is specific to a savings account.
- In the main method of the BankAccountApp class, objects of both Checking and Saving classes are created and stored in a List of type Account. This is possible because Checking and Saving classes are both sub-classes of Account class. This allows us to access and manipulate objects of both types using the methods defined in the Account class.



Discuss each class with detail and screen shots



BankAccountApp class

BankAccountApp class

This is the main class that runs the application. I included some commented-out code to test creating new accounts, which is a good way to test a code.

```
8  import java.util.LinkedList;
9  import java.util.List;
10
11  import static BankAccountApp.utilities.CSV.read;
12
13  ▶ public class BankAccountApp {
14  ▶  public static void main(String[] args) {
15      // TO DO Auto-generate method sub
16
17      // Read a CSV file
18
19      // Test creating new account
20      /*
21      Checking checking1 = new Checking("Dany", "q12345", 99.9);
22
23      Saving saving1 = new Saving("Test", "t000", 0.01);
24
25      checking1.showInfo();
26      System.out.println("*****");
27      saving1.showInfo();
28
29      System.out.println("*****");
30      saving1.deposit(100);
31      saving1.withdraw(9);
32      saving1.transfer("Myself", 0.00001);
33      saving1.compound();
34      */
35  }
```

BankAccountApp class

- In this class, I have the main() method that reads the CSV file, creates a list of accounts, and then iterates over the list to show the account information for each account.

```
36 List<Account> accounts = new LinkedList<Account>();
37
38 // Read a CSV file
39 String file = "/Users/dany/Documents/GitHub/projects-in-bife/BankAccount/src/BankAccountApp/utilities/NewBankAccounts.csv";
40 List<String[]> newAccountHolders = read(file);
41
42 for (String[] accountHolder : newAccountHolders) {
43     String name = accountHolder[0];
44     String ppsn = accountHolder[1];
45     String accountType = accountHolder[2];
46     double initDeposit = Double.parseDouble(accountHolder[3]);
47     //System.out.println("\n\tACCOUNT");
48     //System.out.printf("Name: %s\nPPS Number: %s\nAccount Type: %s\nBalance: %f$\n", name, ppsn, accountType, initDeposit);
49
50     if (accountType.equalsIgnoreCase( anotherString: "Savings")) {
51         //System.out.println("OPEN SAVING ACCOUNT");
52         accounts.add(new Saving(name, ppsn, initDeposit));
53     } else if (accountType.equalsIgnoreCase( anotherString: "Checking")) {
54         //System.out.println("OPEN CHECKING ACCOUNT");
55         accounts.add(new Checking(name, ppsn, initDeposit));
56     } else {
57         System.out.println("ERROR READING ACCOUNT TYPE");
58     }
59 }
60 // Access an element in a data structure
61 //accounts.get(5).showInfo();
62
63 // Access elements in a data structure for all of them
64 for (Account account : accounts) {
65     account.showInfo();
66     System.out.println("*****");
67 }
68 }
69 }
```

Account class

This is an abstract class that serves as the parent class for the Saving and Checking subclasses. In this class, I have defined the common properties and methods that are shared by both subclasses. I have also implemented the IBaseRate interface to include the `setRate()` method.

Account class

This class contains the following properties:

- name: A string that represents the name of the account holder.
- ppsNumber: A string that represents the PPS number of the account holder.
- balance: A double that represents the current balance of the account.
- accountNumber: A string that represents the account number of the account.
- index: An integer that is used to generate unique account numbers.
- rate: A double that represents the interest rate for the account.

```
8 public abstract class Account implements IBaseRate {  
9     // List common properties for savings and checking  
10    2 usages  
11    private String name;  
12    4 usages  
13    private String ppsNumber;  
14    8 usages  
15    private double balance;  
16    6 usages  
17    protected String accountNumber;  
18    2 usages  
19    private static int index = 10000;  
20    4 usages  
21    protected double rate;
```

Account class

Constructor

```
17 // Constructor to set base properties to initialize the account
    2 usages
18 public Account(String name, String ppsNumber, double initDeposit) {
19     this.name = name;
20     this.ppsNumber = ppsNumber;
21     this.balance = initDeposit;
22     // Set account number
23     this.accountNumber = setAccountNumber();
24
25     // Add property baseRate
26     setRate();
27 }
28
```

Account class

This class contains the following methods:

- `setAccountNumber()`: A private method that generates a unique account number using the last two digits of the account holder's PPS number, an index number, and a random number.
- `deposit(double amount)`: A public method that adds the specified amount to the account balance.
- `withdraw(double amount)`: A public method that subtracts the specified amount from the account balance.
- `transfer(String toWhere, double amount)`: A public method that adds the specified amount to the account balance and prints a message indicating the transfer.
- `printBalance()`: A public method that prints the current balance of the account.

```
29 // List common methods
30 1 usage
31 @ private String setAccountNumber() {
32     String lastTwoOfPPS = ppsNumber.substring(ppsNumber.length()-2, ppsNumber.length());
33     index++;
34     int uniqueId = index;
35     int randomNumber = (int) (Math.random() * Math.pow(10, 3));
36     return lastTwoOfPPS + uniqueId + randomNumber;
37 }
38 public void deposit(double amount) {
39     balance += amount;
40     System.out.printf("\nDeposit: %f$", amount);
41     printBalance();
42 }
43 public void withdraw(double amount) {
44     balance -= amount;
45     System.out.printf("\nWithdraw: %f$", amount);
46     printBalance();
47 }
48 public void transfer(String toWhere, double amount) {
49     balance += amount;
50     System.out.printf("\nTransfer: %f$ | To: %s", amount, toWhere);
51     printBalance();
52 }
53 4 usages
54 public void printBalance() {
55     System.out.printf("\nYour Balance is now: %f$", balance);
56 }
57
58
```

Account class

- `compound()`: A public method that calculates the interest accrued on the account balance and adds it to the account balance.
- `showInfo()`: A public abstract method that displays all the information about the account.
- And `SetRate()`;

```
60 // Compound interest rate
61 public void compound() {
62     double accruedInterest = balance * (rate/100);
63     balance += accruedInterest;
64     System.out.printf("\nAccrued Interest: %f$", accruedInterest);
65     printBalance();
66 }
67
68 // Created a method that will display all the information about the class
69 // 3 usages 2 overrides
69 public void showInfo() {
70     System.out.println("\n\tACCOUNT");
71     System.out.format("NAME: %s\nACCOUNT NUMBER: %s\nBALANCE: %s$\nRATE: %f%%\n",
72                       name, accountNumber, balance, rate);
73 }
74
75 // 1 usage 2 implementations
75 public abstract void setRate();
76 }
```

Saving class

This is a subclass of the Account class that represents a savings account.
In this class, I have defined the properties and methods specific to the savings account

Saving class

This class contains the following properties:

- **safetyDepositBoxID**: An integer that represents the ID of the safety deposit box assigned to the account holder.
- **safetyDepositBoxKey**: An integer that represents the key to the safety deposit box assigned to the account holder.

```
8      public class Saving extends Account {  
9          // List properties specific to the Saving account  
10         2 usages  
11         private int safetyDepositBoxId;  
12         2 usages  
13         private int safetyDepositBoxKey;
```

Saving class

Constructor

```
13      // Constructors to initialise Saving account properties
14      1 usage
15      public Saving(String name, String ppsNumber, double initDeposit) {
16          super(name, ppsNumber, initDeposit);
17          accountNumber = '1' + accountNumber;
18
19          // Call the method for safety deposit box
20          setSafetyDepositBox();
21      }
```


Saving class

This class contains the following methods:

- `setRate()`: An overridden method that sets the interest rate for the savings account.
- `showInfo()`: An overridden method that displays all the information about the savings account.
- `setSafetyDepositBox()`: generate random deposit box id(3 number) and box key(4 number)

```
22 // List any methods specific to the Saving account
23 1 usage
24 @Override
25 public void setRate() {
26     rate = getBaseRate() - 0.15;
27 }
28
29 1 usage
30 private void setSafetyDepositBox() {
31     // Safety deposit box ID is a three digit number - random
32     safetyDepositBoxId = (int) (Math.random() * Math.pow(10, 3));
33     safetyDepositBoxKey = (int) (Math.random() * Math.pow(10, 4));
34 }
35
36 3 usages
37 public void showInfo() {
38     super.showInfo();
39     System.out.format("ACCOUNT TYPE: Saving\n" +
40         "Your Savings account Features\n" +
41         "Safety Deposit Box ID: %s\n" +
42         "Safety Deposit Box Key: %s\n",
43         safetyDepositBoxId, safetyDepositBoxKey);
44 }
45 }
```


Checking class

This is a subclass of the Account class that represents a checking account. In this class, I have defined the properties and methods specific to the checking account.

Checking class

This class contains the following properties:

- **cardNumber**: An integer that represents the debit card number assigned to the account holder.
- **cardPIN**: An integer that represents the PIN for the debit card assigned to the account holder.

```
8      public class Checking extends Account {  
9          // List properties specific to the Checking account  
10         2 usages  
11         private int cardNumber;  
12         2 usages  
13         private int cardPIN;  
14     }
```

Checking class

This class contains the following methods:

- `setRate()`: An overridden method that sets the interest rate for the checking account.
- `setDebitCardNumber()`: A private method that generates a unique debit card number for the account holder.
- `showInfo()`: An overridden method that displays all the information about the checking account.

```
22 // List any methods specific to the Checking account
23 1 usage
24 @Override
25 public void setRate() {
26     rate = getBaseRate() * 0.15;
27 }
28
29 1 usage
30 private void setDebitCardNumber() {
31     // Debit card number is a 12 digit number - random
32     cardNumber = (int) (Math.random() * Math.pow(10, 12));
33     cardPIN = (int) (Math.random() * Math.pow(10, 4));
34 }
35
36 3 usages
37 public void showInfo() {
38     super.showInfo();
39     System.out.format("ACCOUNT TYPE: Checking\n" +
40         "Your Checking account Features\n" +
41         "Debit Card Number: %s\n" +
42         "Debit Card's PIN: %s\n",
43         cardNumber, cardPIN);
44 }
```

IBaseRate interface

IBaseRate interface

The IBaseRate interface seems to define a common contract for different types of interest rates. It includes a single method `getBaseRate` that returns a double value representing the current base rate. This interface can be implemented by various classes that calculate the base rate using different algorithms or data sources, allowing the code to be more flexible and adaptable to different scenarios.

```
1 package BankAccountApp.main;  
2  
3 public interface IBaseRate {  
4     // Write a method that returns the base rate  
5     // 2 usages  
6     default double getBaseRate() {  
7         // Input a benchmark rate  
8         return 2.5;  
9     }  
10 }
```

CSV class

The CSV class can be used in many scenarios where data needs to be imported from a CSV file, such as loading financial data or customer information into a program. However, it's worth noting that there are many third-party libraries available for reading and parsing CSV files, which might offer more features and better performance than a custom implementation. [CSV discuss](#)

CSV class

The CSV class seems to be responsible for reading and parsing CSV (Comma-Separated Values) files. It includes a constructor that takes a file path as input and opens the file using a `FileReader` and a `BufferedReader`. It also includes a method `read` that reads the file line by line and returns an array of strings representing the values in each line, split by commas.

```
8 import java.io.BufferedReader;
9 import java.io.FileNotFoundException;
10 import java.io.FileReader;
11 import java.io.IOException;
12 import java.util.LinkedList;
13 import java.util.List;
14
15 // 1 usage
16 public class CSV {
17     // This function will read data from a CSV file and return as a list
18     // 2 usages
19     @ public static List<String[]> read(String file) {
20         List<String[]> data = new LinkedList<>();
21         String dataRow;
22
23         try {
24             BufferedReader bufferedReader = new BufferedReader(new FileReader(file));
25             // As don't know how long the record could be
26             while ((dataRow = bufferedReader.readLine()) != null) {
27                 String[] dataRecord = dataRow.split(regex: ",");
28                 data.add(dataRecord);
29             }
30         } catch (FileNotFoundException e) {
31             System.out.println("COULD NOT FIND FILE");
32             e.printStackTrace();
33         } catch (IOException e) {
34             System.out.println("COULD NOT READ FILE");
35             e.printStackTrace();
36         }
37         return data;
38     }
39 }
```

Testing of parts of the App

video

BankAccount

src

BankAccountApp

main

Project

BankAccountApp.java

Account.java

Checking.java

Saving.java

CSV.java

IBaseRate.java

Project

Commit

Structure

Bookmarks

BankAccount

~\Documents\GitHul

> .idea

> out

> src

> BankAccountApp

> main

Account

BankAccountApp

Checking

IBaseRate

Saving

utilities

CSV

NewBankAccounts.csv

.gitignore

BankAccount.iml

> External Libraries

> Scratches and Consoles

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

```
package BankAccountApp.main;

/*
Created by: Dany
Created on 03/03/2023 : 14:16
Main Application
*/

import java.util.LinkedList;
import java.util.List;

import static BankAccountApp.utilities.CSV.read;

public class BankAccountApp {
    public static void main(String[] args) {
        // TO DO Auto-generate method sub

        // Read a CSV file

        // Test creating new account
        /*
Checking checking1 = new Checking("Dany", "q12345", 99.9);

Saving saving1 = new Saving("Test", "t000", 0.01);

checking1.showInfo();
System.out.println("*****");
saving1.showInfo();

System.out.println("*****");
saving1.deposit(100);
saving1.withdraw(9);
saving1.transfer("Myself", 0.00001);
saving1.compound();
*/

List<Account> accounts = new LinkedList<Account>();
```

10

2

3

Database

Notifications

Git

Run

TODO

Problems

Terminal

Services

Profiler

SonarLint

Build

Build completed successfully in 3 sec, 383 ... (today 09:08)

1:1 LF UTF-8 4 spaces main

Evaluate your learning

- What did I do ?
- What did I learn ?
- What could I do to improve the app in the future ?

What did I do ?

- I have provided the code for a basic bank account management system. In the code, I have created three classes: Account (an abstract class), Checking (a subclass of Account), and Savings (another subclass of Account). The Account class has common properties and methods for both Checking and Savings accounts, while the subclasses have properties and methods specific to each account type.
- I have also created a main class (BankAccountApp) that reads account information from a CSV file and creates new accounts based on that information. Additionally, the main class has methods for depositing, withdrawing, transferring, and printing the balance of each account.

What did I learn ?

From this Skills Demonstraton, I have learned how to create classes and objects, how to use inheritance to create subclasses, and how to read from a CSV file.

What could I do to improve the app in the future ?

To improve the app in the future, I could add additional features such as:

- Overdraft protection for Checking accounts
- ATM functionality
- Online banking portal
- Mobile banking app
- Credit card accounts and transactions
- Investment accounts
- Loan accounts
- International currency support
- Security measures such as two-factor authentication and encryption

It is the end

- Do you have some questions for my?